KR Mangalam University

Assignment – 1

Data Structure

Unit – 1

Foundations & Algorithmic Analysis

Submitted by – Himanshi

B.Tech CSE AI & ML

Section – A

Roll No. – 2501730137

Submitted to – Mrs. Neetu Chauhan Mam

1.Introduction

In this assignment, I built a Python toolkit called Algorithmic Efficiency & Recursion Toolkit

This toolkit includes:

- Stack ADT implementation
- Recursive Factorial
- Recursive Fibonacci (Naive & Memoized)
- Tower of Hanoi
- Recursive Binary Search
- Complexity and paradigm analysis

2. Stack ADT

A Stack ADT (Abstract Data Type) is a linear data structure that follows the LIFO (Last In, First Out) principle, where the last element inserted into the stack is the first element to be removed.

It supports basic operations such as push (insert), pop (remove), and peek (view top element).

Why Stack Operations Are O(1)?

Stack operations like:

- push
- pop
- peek

are **O(1)** (constant time) because they only work on the top element.

They do not depend on the size of the stack.

3. Factorial (Recursive)

Recursive factorial is a method of calculating the factorial of a number using recursion, where the function calls itself until it reaches a base case.

Factorial of a number n (written as n!) is defined as:

$$n! = n \times (n-1)!$$

Base Case:

$$0! = 1$$

4. Fibonacci

Naive Recursive Fibonacci

fib(n) = fib(n−1) + fib(n−2)

Time Complexity

- Time Complexity: $O(2^n)$
- Space Complexity**:** O(n)

Why It Is Slow?

Naive Fibonacci repeats the same calculations many times.

Example:
fib(5) calculates fib(3) multiple times.

Because of repeated calls, the number of recursive calls grows exponentially.

5. Memoized Fibonacci (Dynamic Programming)

Memoization stores already computed values.

If fib(5) is already computed, we reuse it instead of recalculating.

Time Complexity

- Time Complexity: O(n)
- Space Complexity: O(n)

Big-O, Big-Ω, Big-Θ

- Best Case: O(n)
- Worst Case: O(n)
- Θ(n)

Memoization improves efficiency by avoiding repeated work.

6. Tower of Hanoi

Tower of Hanoi follows recursion:

1. Move n−1 disks to auxiliary
2. Move largest disk
3. Move n−1 disks to destination

Why Moves = $2^n - 1$?

For:
n = 1 → 1 move
n = 2 → 3 moves
n = 3 → 7 moves

Pattern:
Moves(n) = 2 × Moves(n−1) + 1

Solving this gives:
Moves = $2^n - 1$

Complexity

- Time Complexity: $O(2^n)$
- Space Complexity: O(n) (recursion depth)

7. Recursive Binary Search

Binary Search works on sorted arrays.

It divides the array into half at every step.

Recurrence relation:

$T(n) = T(n/2) + O(1)$

Solving this gives:

Time Complexity = O(log n)

Best / Average / Worst Cases

- Best Case: O(1) (key found in middle)
- Average Case: O(log n)
- Worst Case: O(log n)

Space Complexity:

- O(log n) (recursion stack)

8. Paradigm Reflection

1. Binary Search → Divide & Conquer

Binary Search divides the array into two halves and solves only one half. Divide and conquer is a technique where a problem is divided into smaller parts, each part is solved separately, and then the results

are combined. In Binary Search, the array is divided in two halves again and again. Instead of searching the whole array, we only search in one half each time. This reduces the problem size quickly and makes the algorithm efficient.

Real life example - Searching a name in a sorted contact list by checking the middle and eliminating half of the list each time is a real-life example of Divide & Conquer.

2. Memoized Fibonacci → Dynamic Programming

Dynamic Programming:

- Solves overlapping subproblems
- Stores previous results

Fibonacci has overlapping subproblems.
Memoization avoids repeated work.

Real life example - Real-life examples of Dynamic Programming are remembering previously solved calculations and reusing them, like saving calculator results or Google Maps storing shortest routes to avoid repeated computation.

3. Tower of Hanoi → Recursion

Tower of Hanoi:

- Has a base case (n = 1)
- Solves smaller problem (n−1)
- Combines solution

Real life example - Moving stacked plates from one shelf to another by first moving smaller plates and then the largest plate.

9. Conclusion

In this assignment, I learned how recursion works using base cases and smaller subproblems. I understood the difference between naive and optimized algorithms, especially how memoization improves Fibonacci efficiency. I also learned how to analyze time and space complexity using Big-O notation and how algorithms follow different paradigms like Divide & Conquer, Dynamic Programming, and Recursion. This assignment improved my understanding of both implementation and algorithm efficiency.